

Assignment - 4.1

aim :

To implement half subtractor using basic gates, XOR gates, NAND gates and NOR gates.

Theory :

A Half Subtractor is a combinational logic circuit used to subtract two single-bit inputs, A (minuend) and B (subtrahend). It produces two outputs: Difference (D) and Borrow (Bout), where Difference is the result bit and Borrow indicates if a borrow is needed.

The logical expressions are :

- $D = A \oplus B$
- $Bout = \bar{A} \cdot B$

The circuit can be implemented using basic gates (AND, OR, NOT), or more efficiently using an XOR gate for Difference and suitable gates for Borrow. Since NAND and NOR are universal gates, the Half Subtractor can also be realised using only NAND or only NOR gates.

VHDL Codes :

Half Subtractor using basic gates :

library ieee;

Teacher's Signature

Expt. No. _____

```
use ieee.std_logic_1164.all;
entity halfsubtractor-basic is
    port (a, b : in std_logic;
          diff, borrow : out std_logic);
end halfsubtractor-basic;
architecture Behavioral of halfsubtractor-basic is
begin
    diff <= ((not a) and b) or ((not b) and a);
    borrow <= (not a) and b;
end Behavioral;
```

Half Subtractor using XOR gate :

```
library ieee;
use ieee.std_logic_1164.all;
entity halfsubtractor-xor is
    port (a, b : in std_logic;
          diff, borrow : out std_logic);
end halfsubtractor-xor;
architecture Behavioral of halfsubtractor-xor is
begin
    diff <= a xor b;
    borrow <= (not a) and b;
end Behavioral;
```

Half Subtractor using NAND gates :

```
library ieee;
use ieee.std_logic_1164.all;
```

Teacher's Signature _____


```
entity halfsubtractor - nand is
port (a, b : in std_logic;
      diff, borrow : out std_logic);
end halfsubtractor - nand;
architecture Behavioral of halfsubtractor - nand is
begin
    diff <= (a nand (a nand b)) nand (b nand (
a nand b));
    borrow <= ((a nand a) nand b) nand ((a
nand a) nand b);
end Behavioral;
```

Half Subtractor using NOR gates :

```
library ieee;
use ieee.std_logic_1164.all;
entity halfsubtractor - nor is
port (a, b : in std_logic;
      diff, borrow : out std_logic);
end halfsubtractor - nor;
architecture Behavioral of halfsubtractor - nor is
begin
    diff <= ((a nor a) nor (b nor b)) nor ((a nor b)
nor (a nor b));
    borrow <= ((a nor a) nor b) nor ((a nor a) nor
b);
end Behavioral;
```


Assignment - 4.2

Ques:

To implement full subtractor using basic gates, XOR gates, NAND gates and NOR gates.

Theory:

A Full Subtractor is a combinational logic circuit used to subtract three single-bit inputs: A (minuend), B (subtrahend), and Borrow-in (B_{in}). It produces two outputs: Difference (D) and Borrow-out (B_{out}), where Difference represents the result bit and Borrow-out indicates if a borrow is required for the next higher bit.

The logical expressions are:

- $D = A \oplus B \oplus B_{in}$
- $B_{out} = \bar{A}B + B \cdot B_{in} + \bar{A} \cdot B_{in}$

The circuit can be implemented using basic gates (AND, OR, NOT) or more efficiently using XOR gates for Difference and suitable combinations of gates for Borrow. Since NAND and NOR are universal gates, the Full Subtractor can also be realised using only NAND or only NOR gates.

Teacher's Signature _____

VHDL Codes :Full Subtractor using basic gates :

```
library ieee;
```

```
use ieee.std_logic_1164.all;
```

```
entity fullsubtractor_basic is
```

```
port (a, b, bin : in std_logic;
```

```
      diff, bout : out std_logic);
```

```
end fullsubtractor_basic;
```

```
architecture Behavioral of fullsubtractor_basic is
```

```
begin
```

```
    diff <= ((not a) and (not b) and bin)
```

```
           or ((not a) and b and (not bin))
```

```
           or (a and (not b) and (not bin))
```

```
           or (a and b and bin);
```

```
    bout <= ((not a) and b)
```

```
           or ((not a) and bin)
```

```
           or (b and bin);
```

```
end Behavioral;
```

Full Subtractor using XOR gate :

```
library ieee;
```

```
use ieee.std_logic_1164.all;
```

```
entity fullsubtractor_xor is
```

```
port (a, b, bin : in std_logic;
```

```
      diff, bout : out std_logic);
```

```
end fullsubtractor_xor;
```

Teacher's Signature _____

architecture Behavioral of fullsubtractor - xor is
begin

```
diff <= a xor b xor bin;  
bout <= ((not a) and b) or ((not a) and bin) or  
((b and bin);  
end Behavioral;
```

Full Subtractor using NAND gates :

```
library ieee;  
use ieee.std-logic-1164.all;  
entity fullsubtractor-nand is  
port (a, b, bin : in std-logic;  
diff, bout : out std-logic);  
end fullsubtractor-nand;
```

architecture Behavioral of fullsubtractor-nand is
signal ab-nand, axorb : std-logic;

begin

```
ab-nand <= a nand b;  
axorb <= (a nand ab-nand) nand (b nand  
ab-nand);  
diff <= (axorb nand (axorb nand bin)) nand  
(bin nand (axorb nand bin));  
bout <= ((a nand a) nand b) nand ((a nand a)  
nand bin) nand (b nand bin);  
end Behavioral;
```


Full Subtractor using NOR gates :

```
library ieee;
use ieee.std_logic_1164.all;
entity fullsubtractor_nor is
port (a, b, bin : in std_logic;
      diff, bout : out std_logic;
end fullsubtractor_nor;
architecture Behavioral of fullsubtractor_nor is
    signal axorb, anor : std_logic;
begin
    anor <= a nor a;
    axorb <= (((anor nor (b nor b)) nor (a nor b))
nor ((anor nor (b nor b)) nor (a nor b)));
    diff <= (((axorb nor axorb) nor (bin nor bin))
nor (axorb nor bin)) nor (((axorb nor axorb)
nor (bin nor bin)) nor (axorb nor bin));
    bout <= ((anor nor b) nor (nor nor b)) nor
((anor nor bin) nor (anor nor bin)) nor ((b nor
bin) nor (b nor bin));
end Behavioral;
```


Assignment-5.1

dim:

To implement 1-bit comparator using VHDL.

Theory:

A 1-bit comparator is a combinational logic circuit used to compare two single-bit inputs, A and B. It produces three outputs: $A > B$, $A = B$, and $A < B$, indicating the relative magnitude of the inputs.

The logical expressions are:

- $A > B = A \cdot \overline{B}$
- $A < B = \overline{A} \cdot B$
- $A = B = A \odot B$

The circuit can be implemented using basic gates (AND, OR, NOT), or more efficiently using an XNOR gate for equality and suitable AND/NOT combinations for the other outputs. Thus, a 1-bit comparator is a basic building block for designing multi-bit comparison circuits in digital systems.

VHDL Code:

```
library ieee;  
use ieee.std_logic_1164.all;  
entity comparator_1bit is  
port (a, b : in std_logic;
```

Teacher's Signature _____

greater, equal, less : out std-logic);
 end comparator-1bit;
 architecture Behavioral of comparator-1bit is
 begin

greater <= a and (not b);
 equal <= (a and b) or ((not a) and (not b));
 less <= (not a) and b;
 end Behavioral;

Assignment - 5.2

Aim :

To implement 2-bit comparator using VHDL.

Theory :

A 2-bit comparator is a combinational logic circuit used to compare two 2-bit binary numbers, $A = A_1 A_0$ and $B = B_1 B_0$. It produces three outputs: $A > B$, $A = B$, and $A < B$, indicating the relative magnitude of the two numbers.

The logical expressions are :

- $A = B = (A_1 \odot B_1)(A_0 \odot B_0)$
- $A > B = A_1 \bar{B}_1 + (A_1 \odot B_1) A_0 \bar{B}_0$
- $A < B = \bar{A}_1 B_1 + (A_1 \odot B_1) \bar{A}_0 B_0$

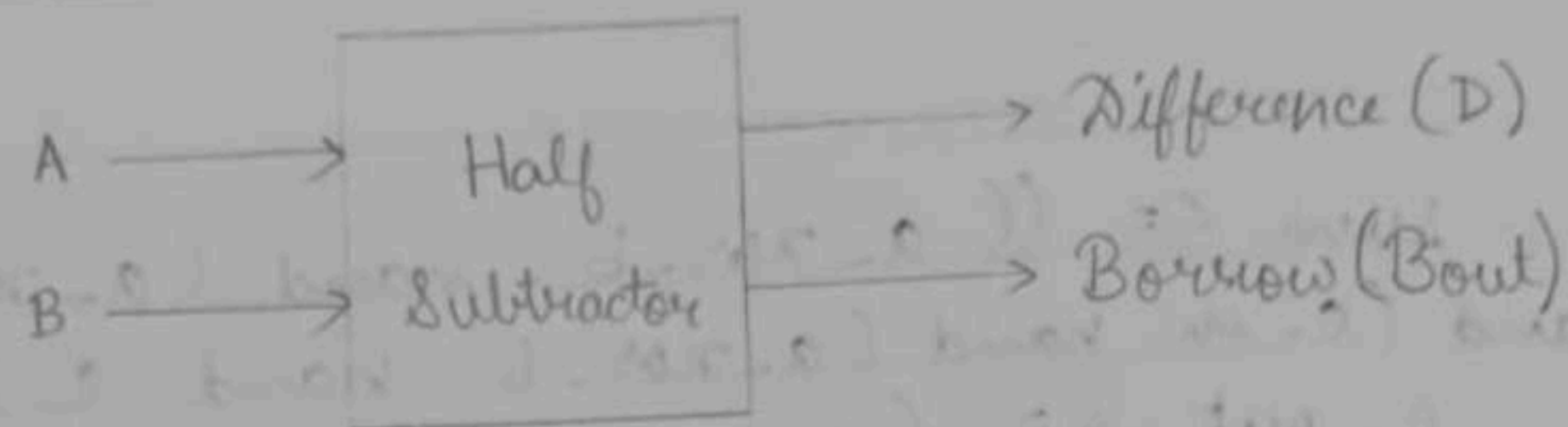
The circuit can be implemented using basic gates (AND, OR, NOT), or efficiently using XNOR gates for equality checking and suitable combinations of AND/OR gates.

Teacher's Signature _____

for magnitude comparison. Thus, a 2-bit comparator extends the concept of 1-bit comparison and serves as a building block for higher-bit comparators in digital systems.

VHDL Code :

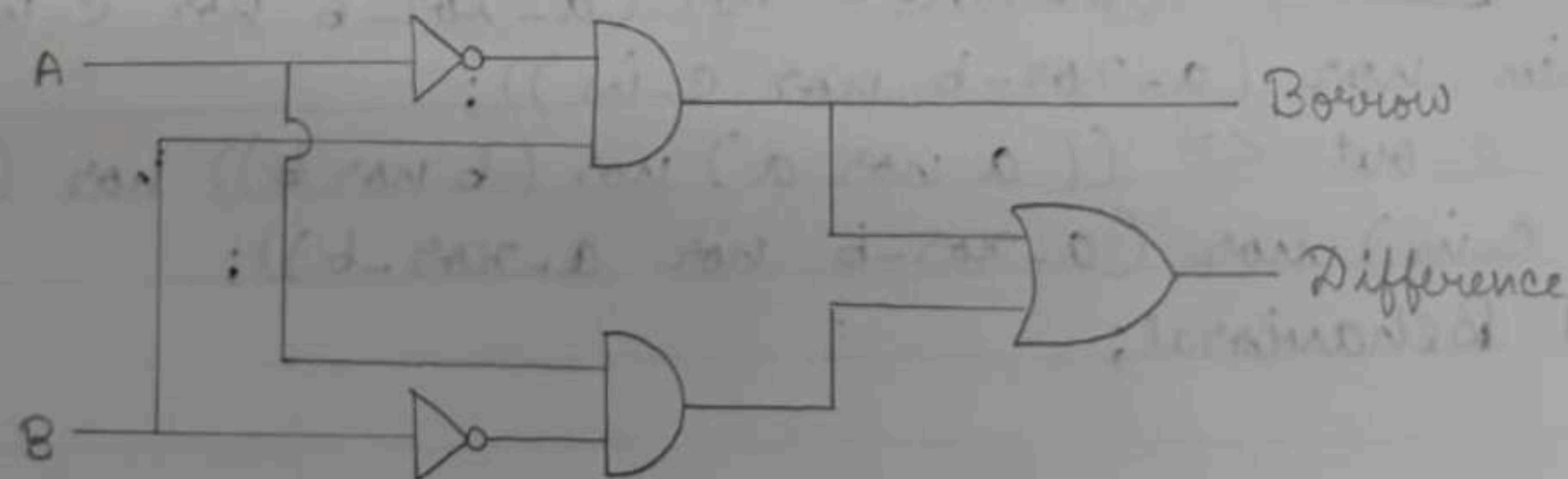
```
library ieee;
use ieee.std_logic_1164.all;
entity comparator_2bit is
port (a1, a0, b1, b0 : in std_logic;
      greater, equal, less : out std_logic);
end comparator_2bit;
architecture Behavioral of comparator_2bit is
begin
    greater <= (a1 and (not b1)) or ((a1 xor b1) and
a0 and (not b0));
    equal <= (a1 xor b1) and (a0 xor b0);
    less <= ((not a1) and b1) or ((a1 xor b1) and
(not a0) and b0);
end Behavioral;
```

Block Diagram of Half Subtractor

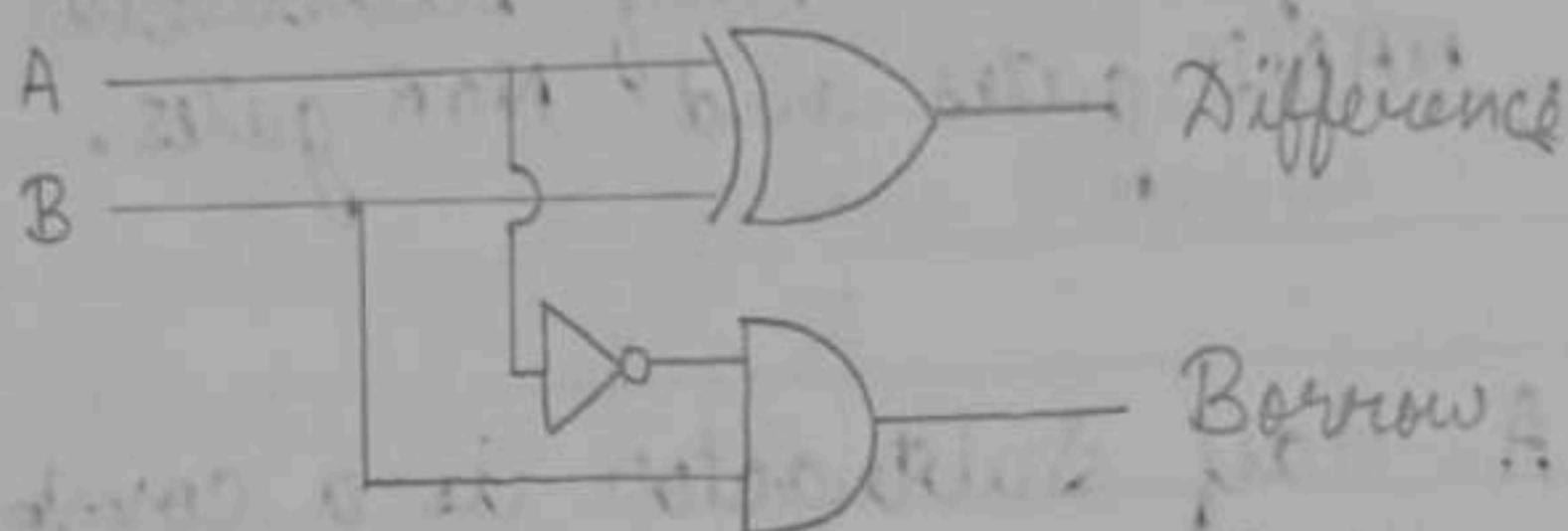
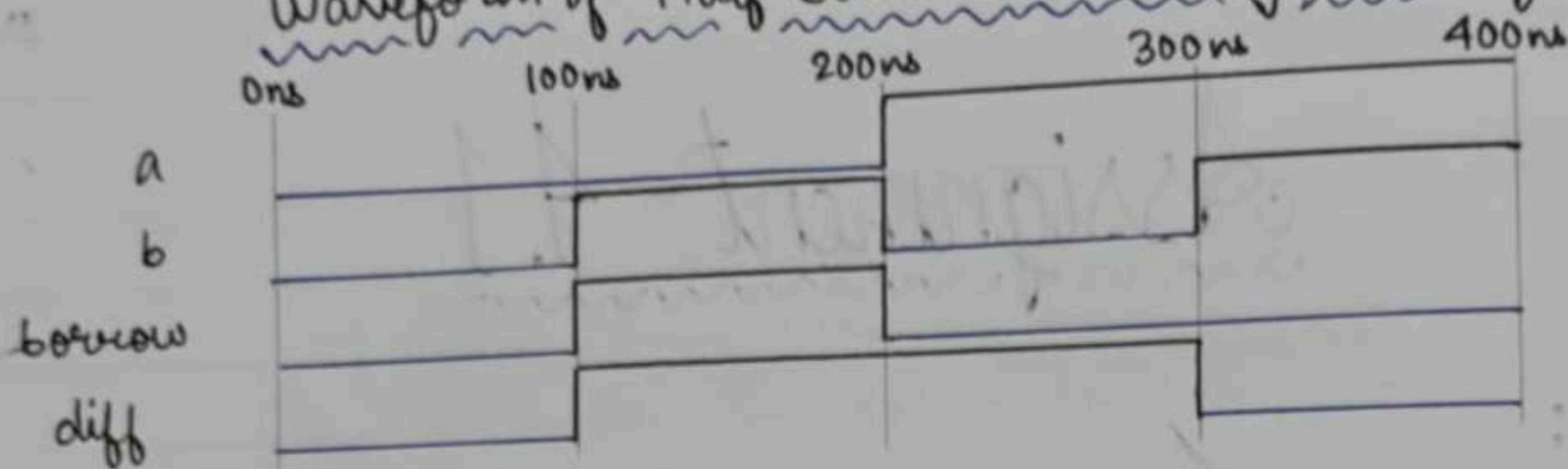
Truth Table of Half Subtractor

A	B	Difference	Borrow
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

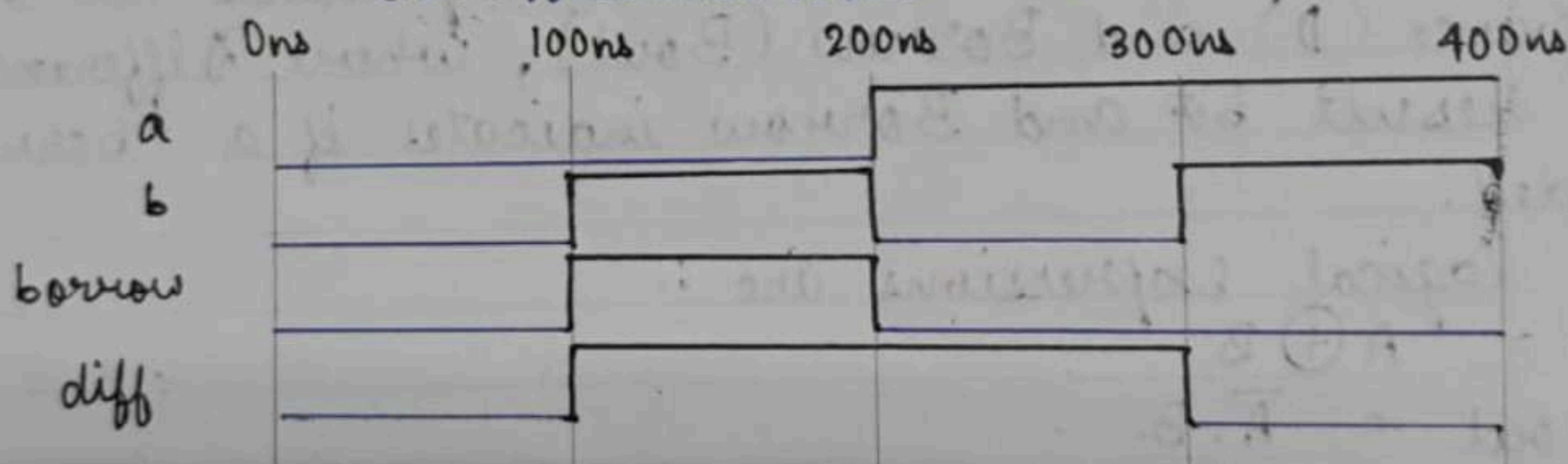


Half Subtractor using basic gates

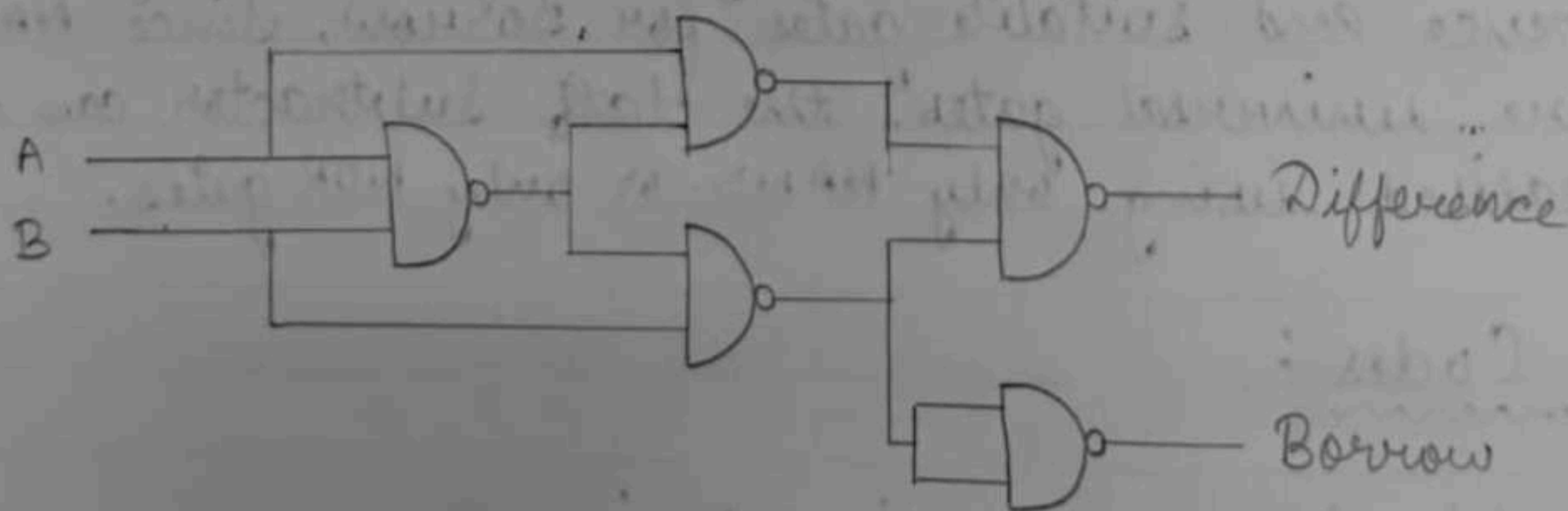
Waveform of Half Subtractor using basic gates



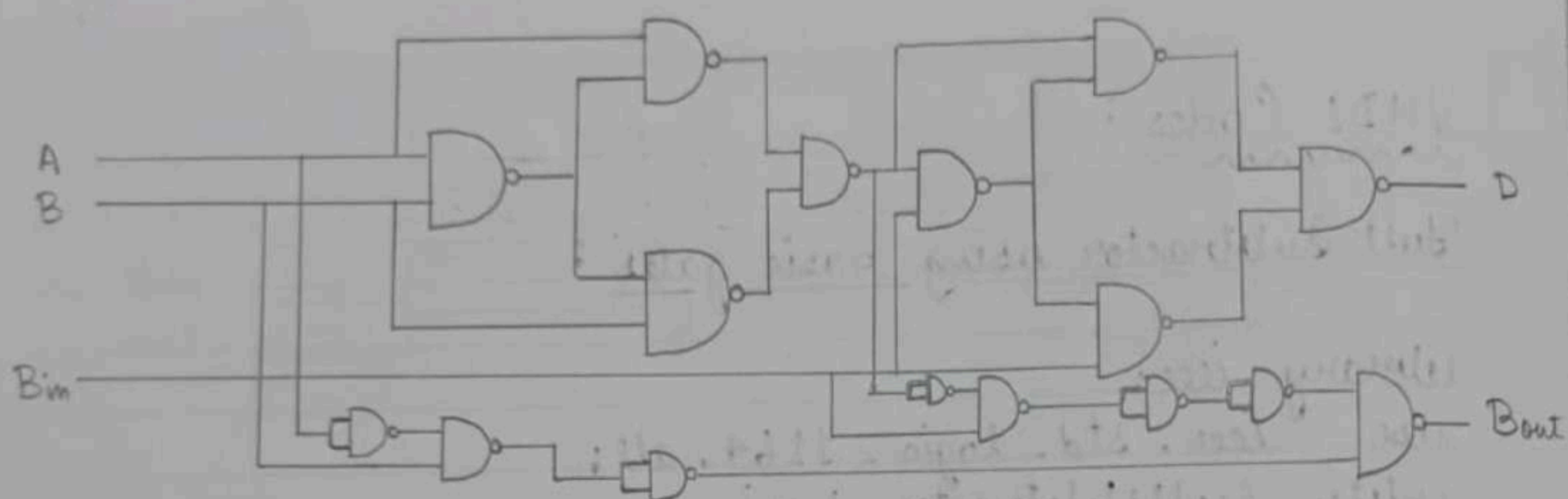
Half Subtractor using XOR gate



Waveform of Half Subtractor using XOR gate

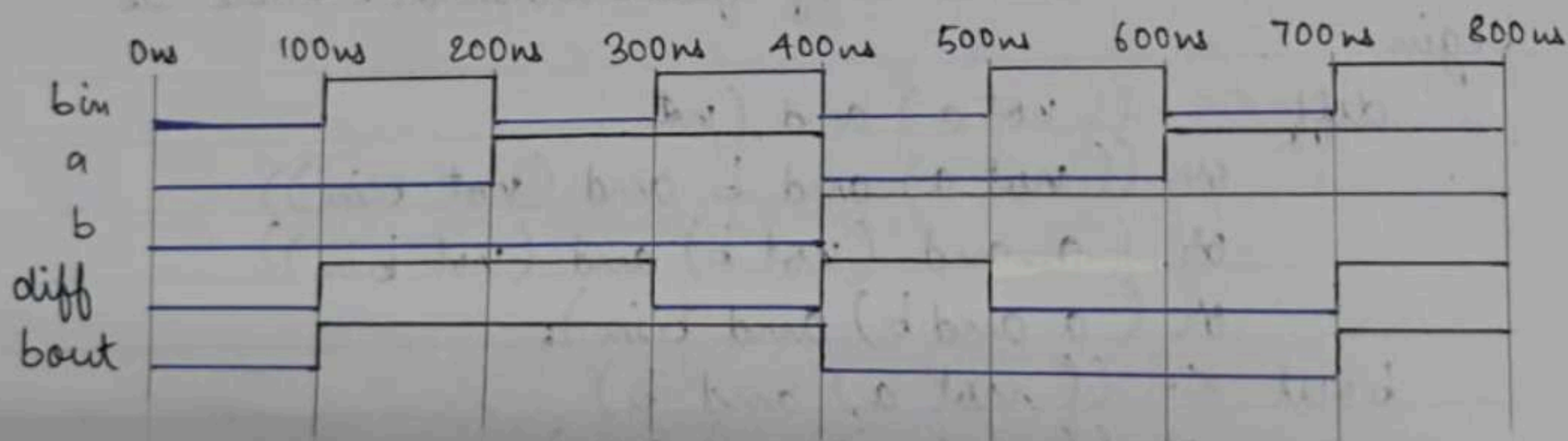


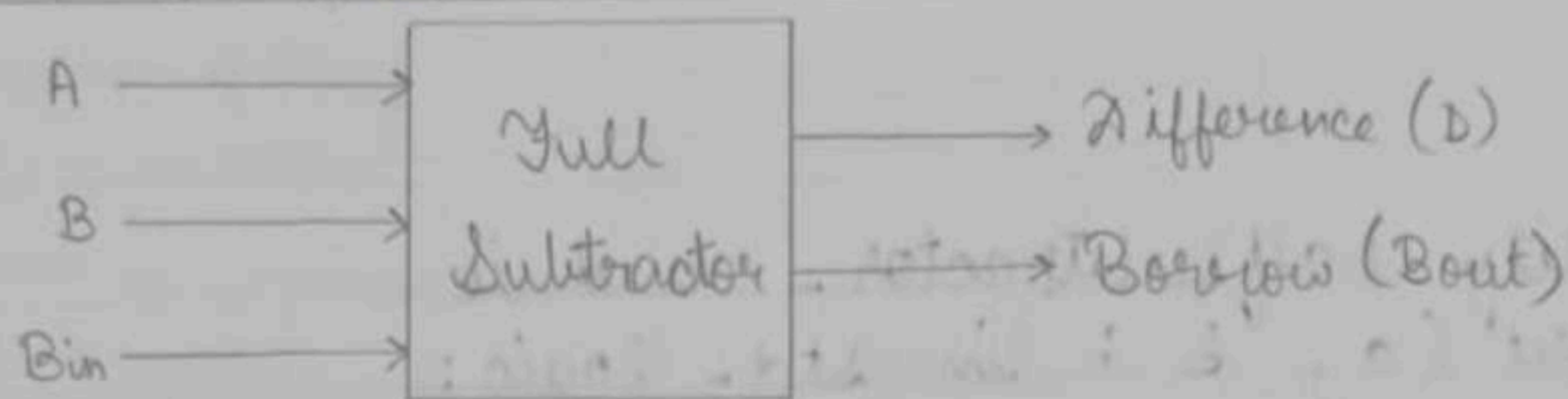
Half Subtractor using NAND gates



Full Subtractor using NAND gates

Waveform of Full Subtractor using NAND gates

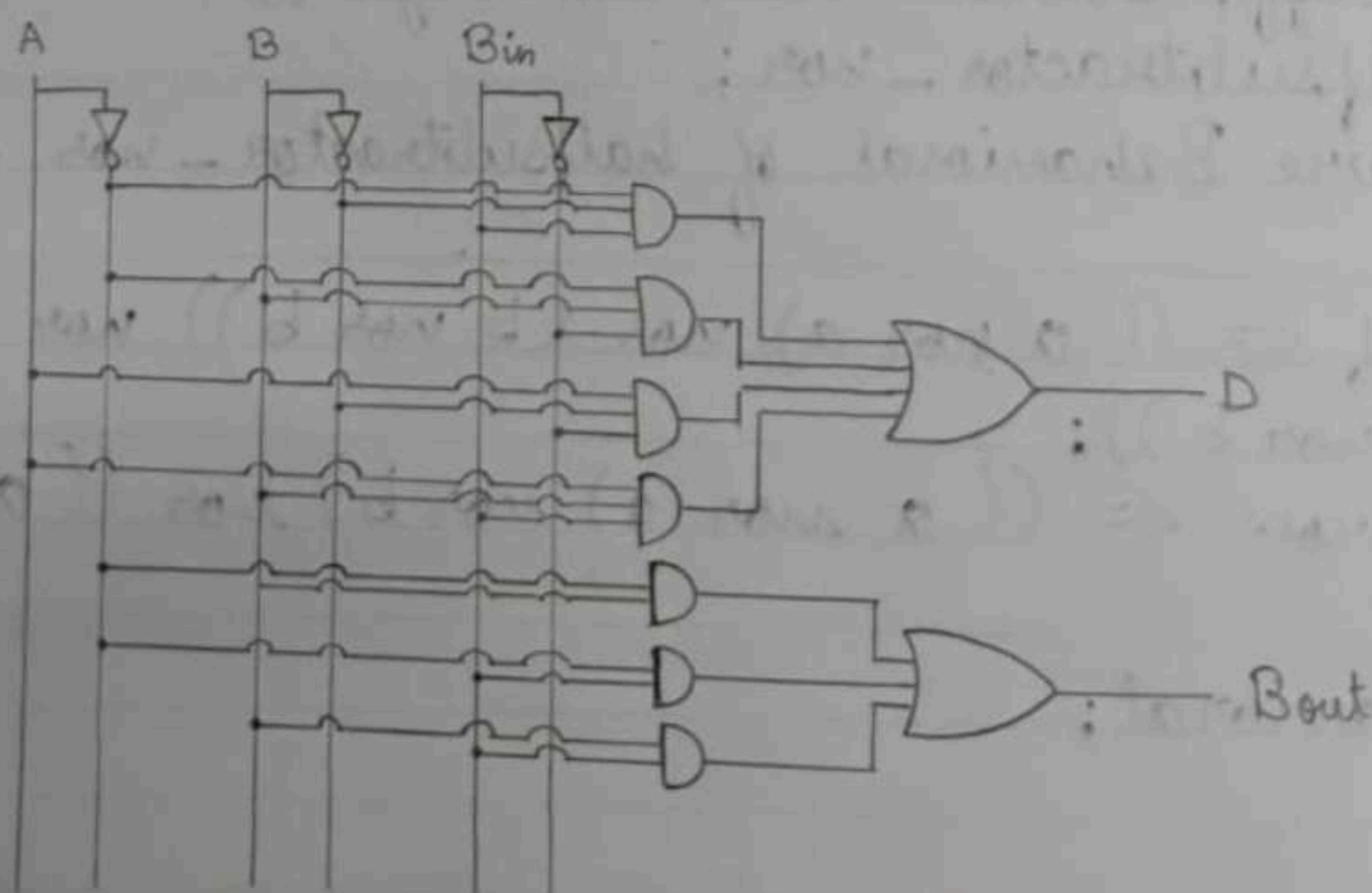




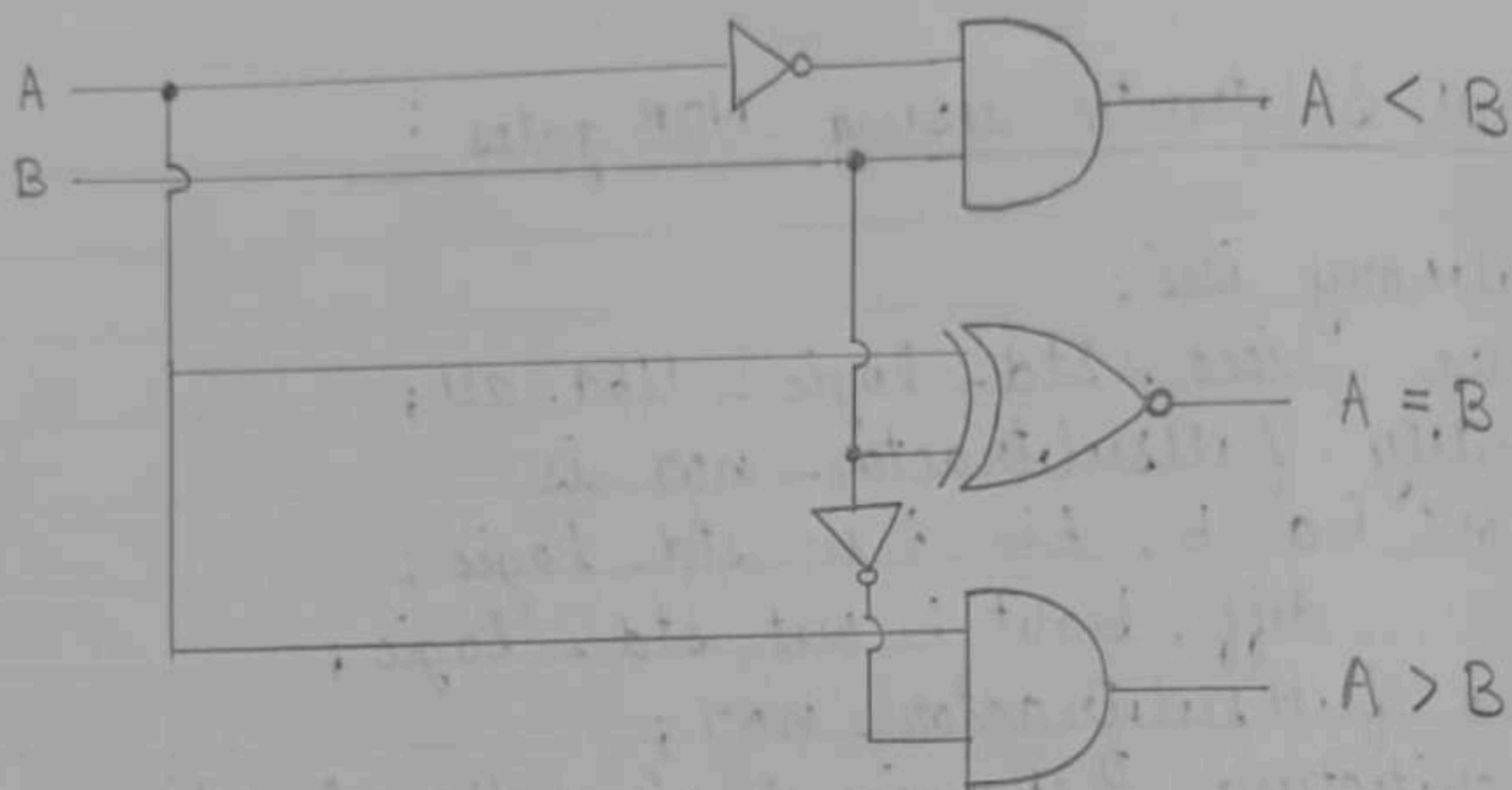
Block Diagram of Full Subtractor

Truth Table of Full Subtractor

A	B	Bin	D	Bout
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1



Full Subtractor using basic gates

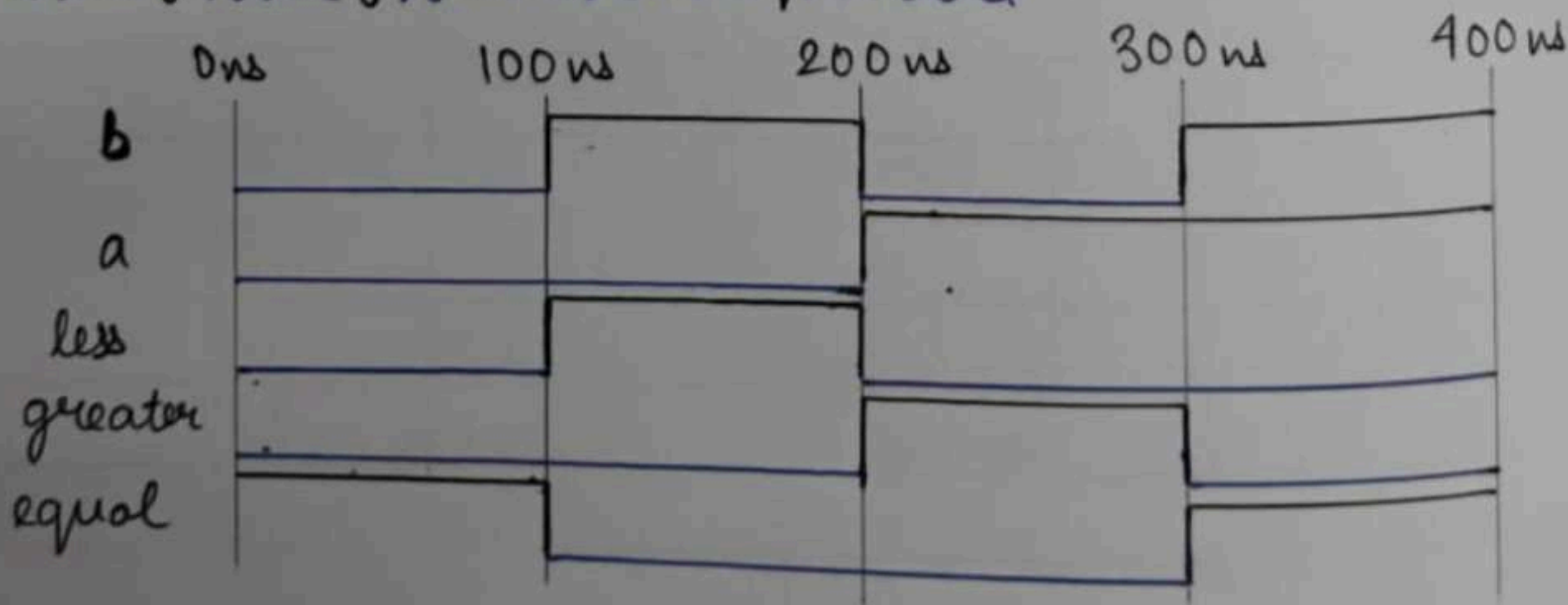


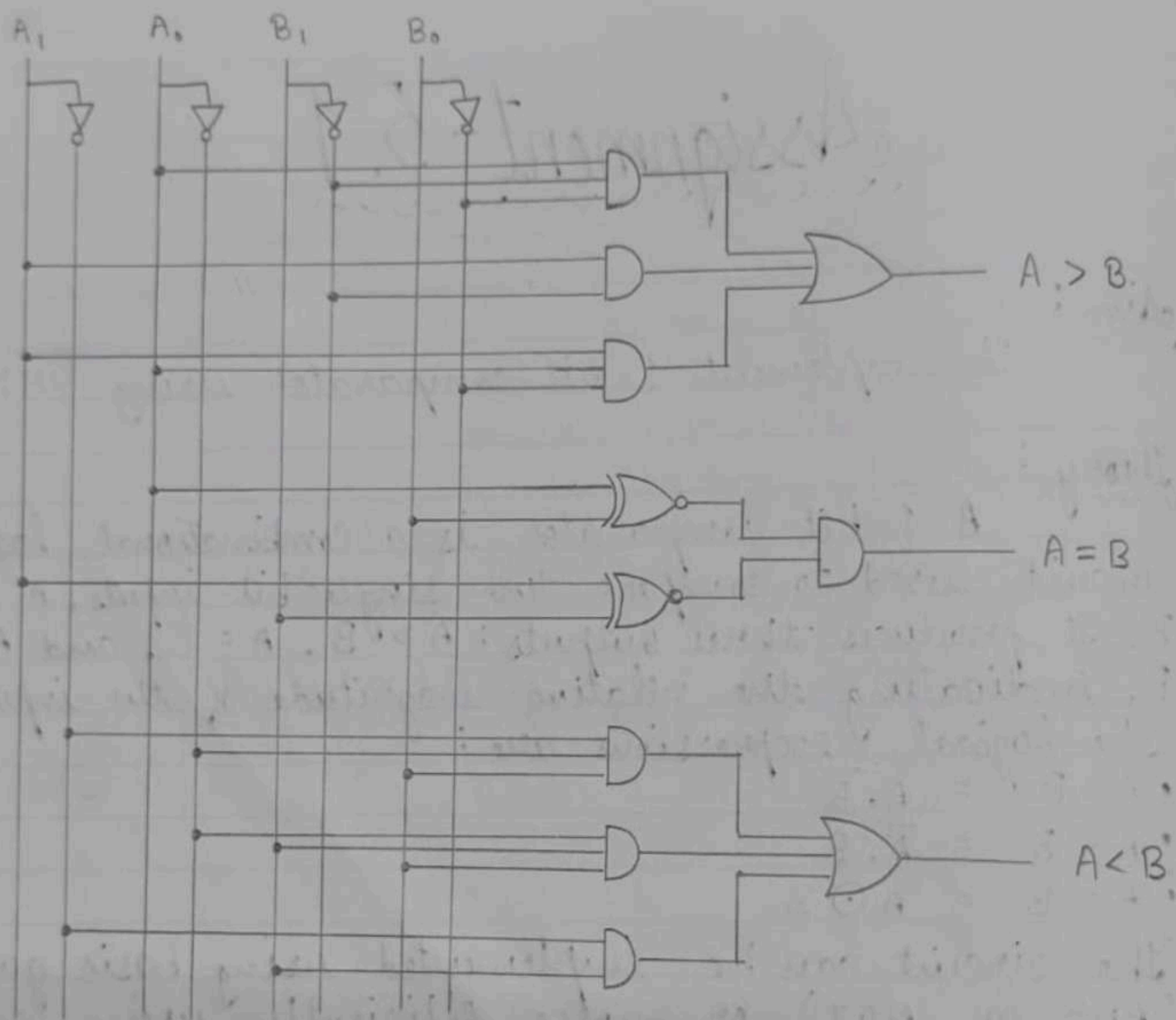
Logic circuit for 1-bit Comparator

Truth Table of 1-bit Comparator

A	B	A < B	A = B	A > B
0	0	0	1	0
0	1	1	0	0
1	0	0	0	1
1	1	0	1	0

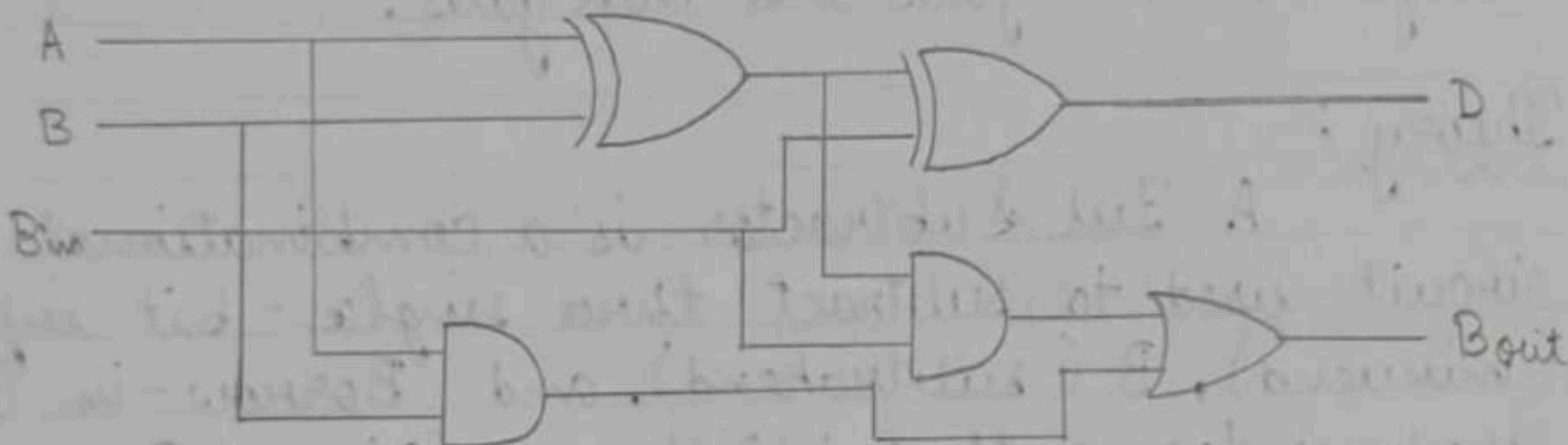
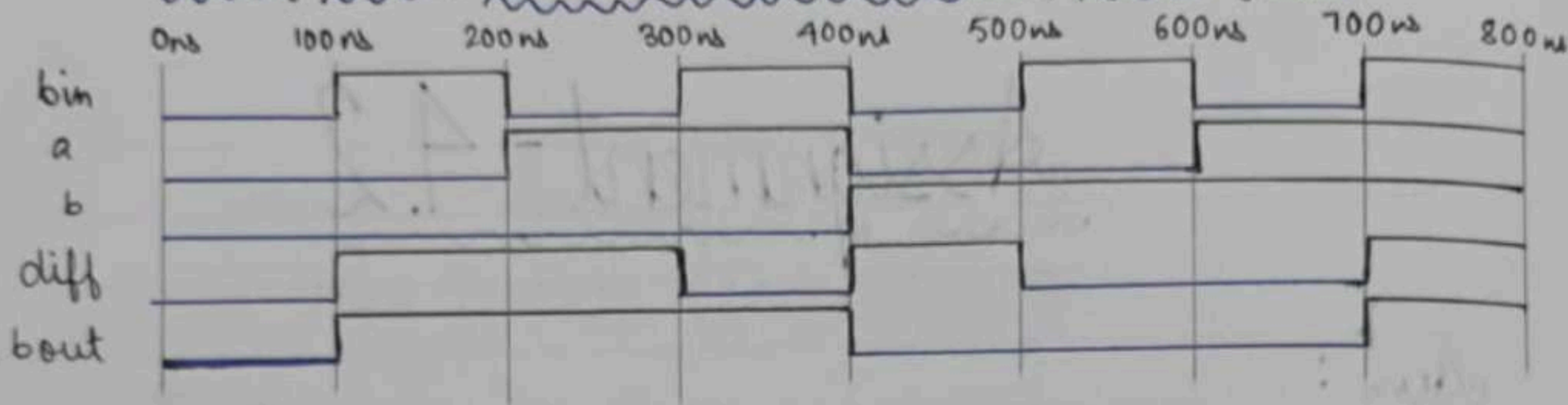
Waveform of 1-bit Comparator





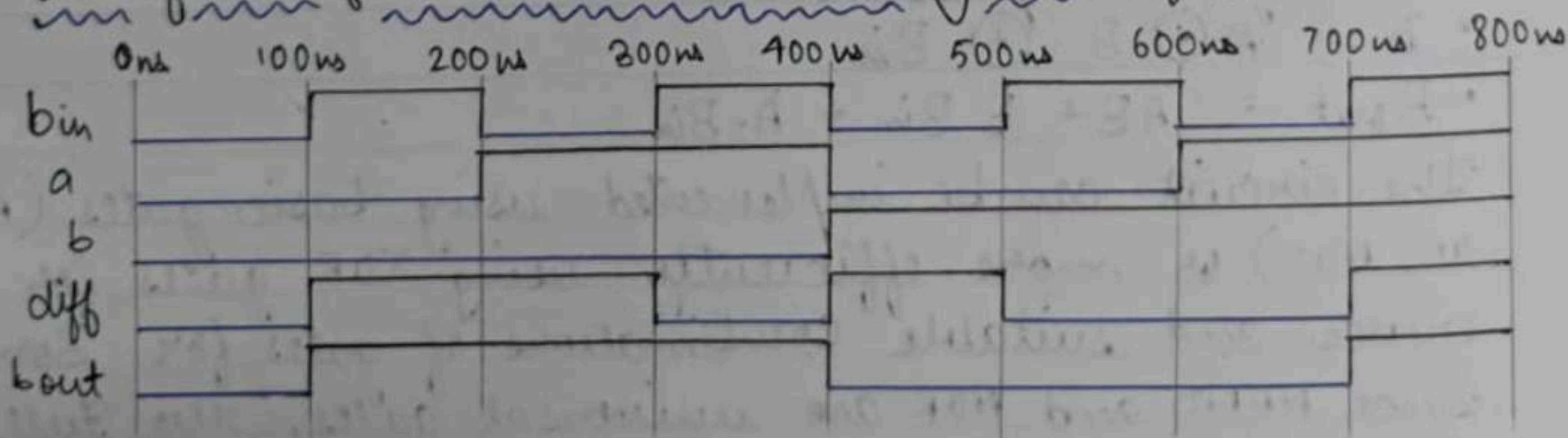
Logic circuit for 2-bit comparator

Waveform of Full Subtractor using basic gates



Full Subtractor using XOR gates

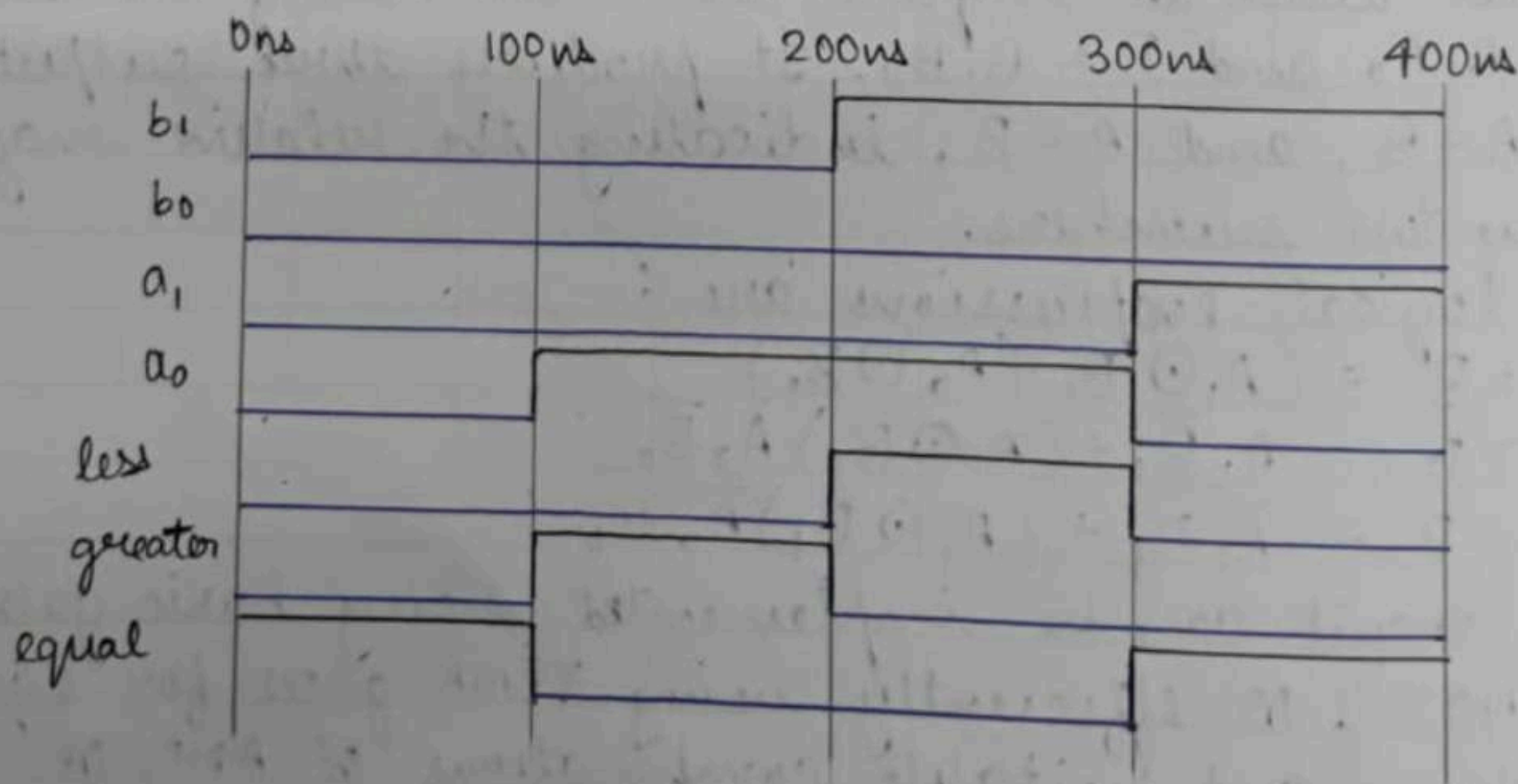
Waveform of Full Subtractor using XOR gates



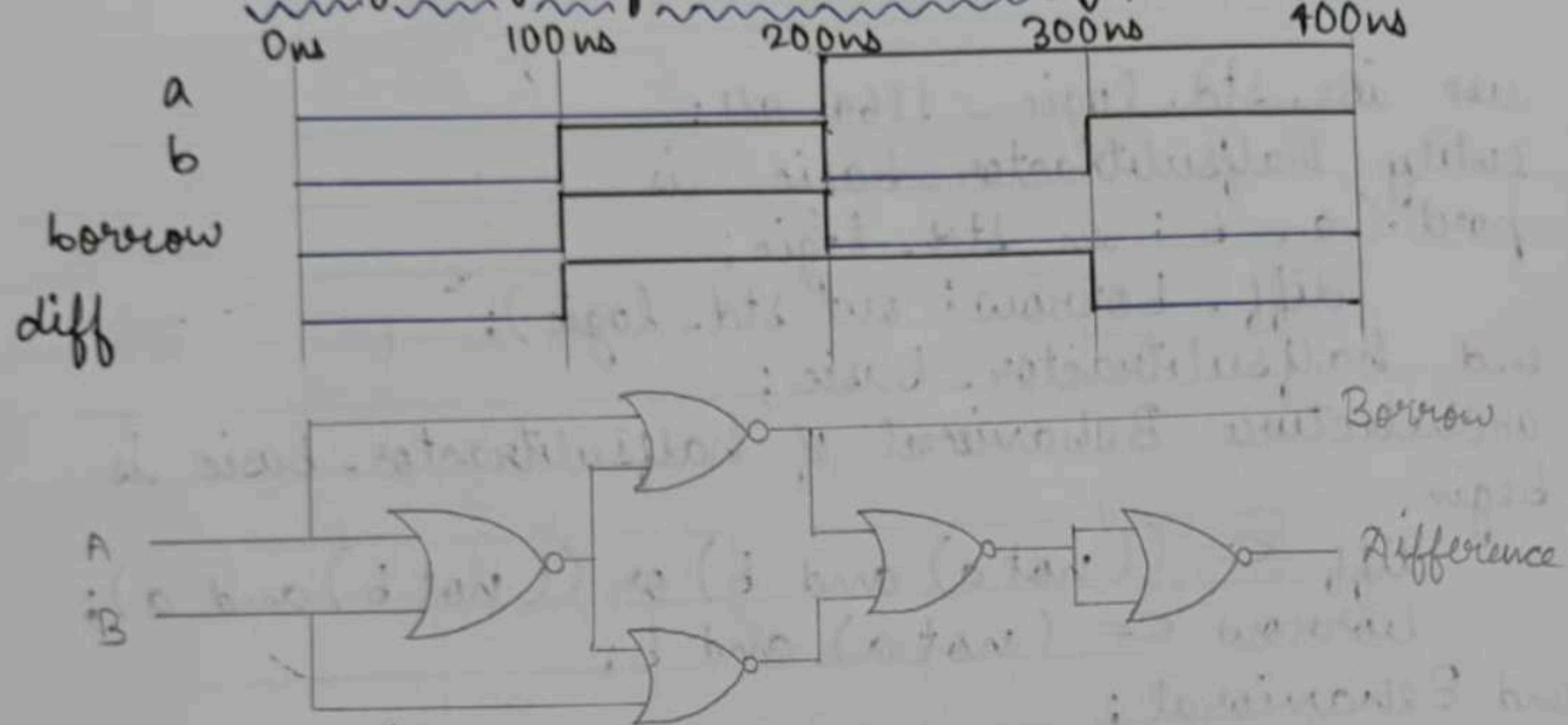
Truth Table of 2-bit Comparator

A ₁	A ₀	B ₁	B ₀	A < B	A = B	A > B
0	0	0	0	0	1	0
0	0	0	1	1	0	0
0	0	1	0	1	0	0
0	0	1	1	1	0	0
0	1	0	0	0	0	1
0	1	0	1	0	1	0
0	1	1	0	1	0	0
0	1	1	1	1	0	0
1	0	0	0	0	0	1
1	0	0	1	0	0	1
1	0	1	0	0	1	0
1	0	1	1	1	0	0
1	1	0	0	0	0	1
1	1	0	1	0	0	1
1	1	1	0	0	0	1
1	1	1	1	0	1	0

Waveform of 2-bit Comparator

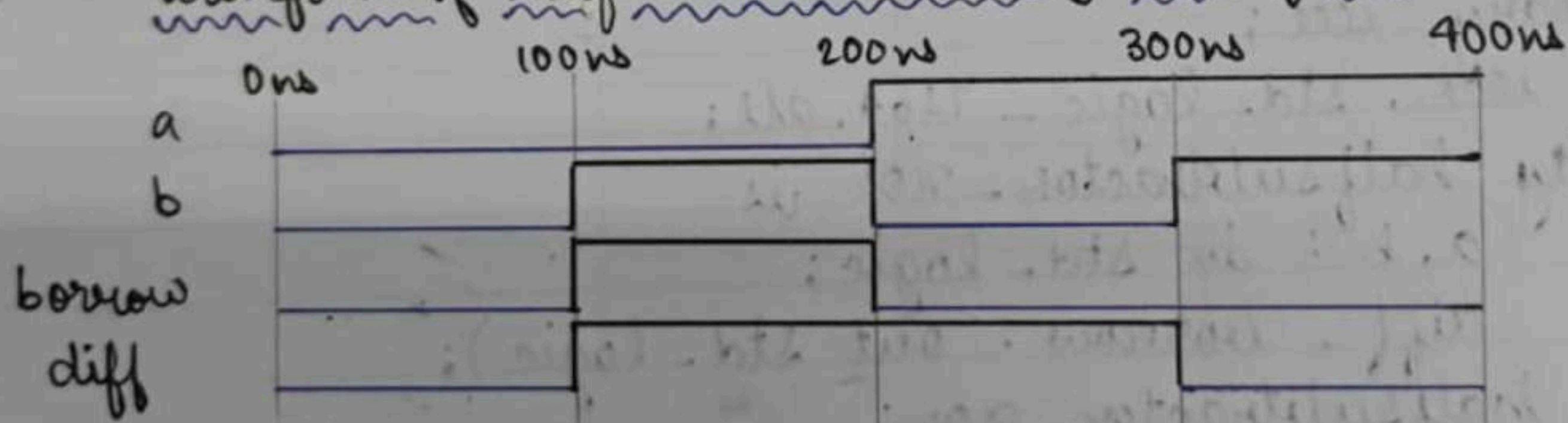


Waveform of Half Subtractor using NAND gates



Half Subtractor using NOR gates

Waveform of Half Subtractor using NOR gates



Subject / Important Note :

Page No. :

Date :

Expt. No. :

1:4 Demultiplexer:-

library IEEE;

use IEEE.STD_Logic_1164.ALL;

entity demux1to4 is

port (I : in STD_Logic;

S0 : in STD_Logic;

S1 : in STD_Logic;

Y0 : out STD_Logic;

Y1 : out STD_Logic;

Y2 : out STD_Logic;

Y3 : out STD_Logic);

end demux1to4;

architecture dataflow of demux1to4 is

signal nS0, nS1 : STD_Logic;

begin

nS0 <= NOT S0;

nS1 <= NOT S1;

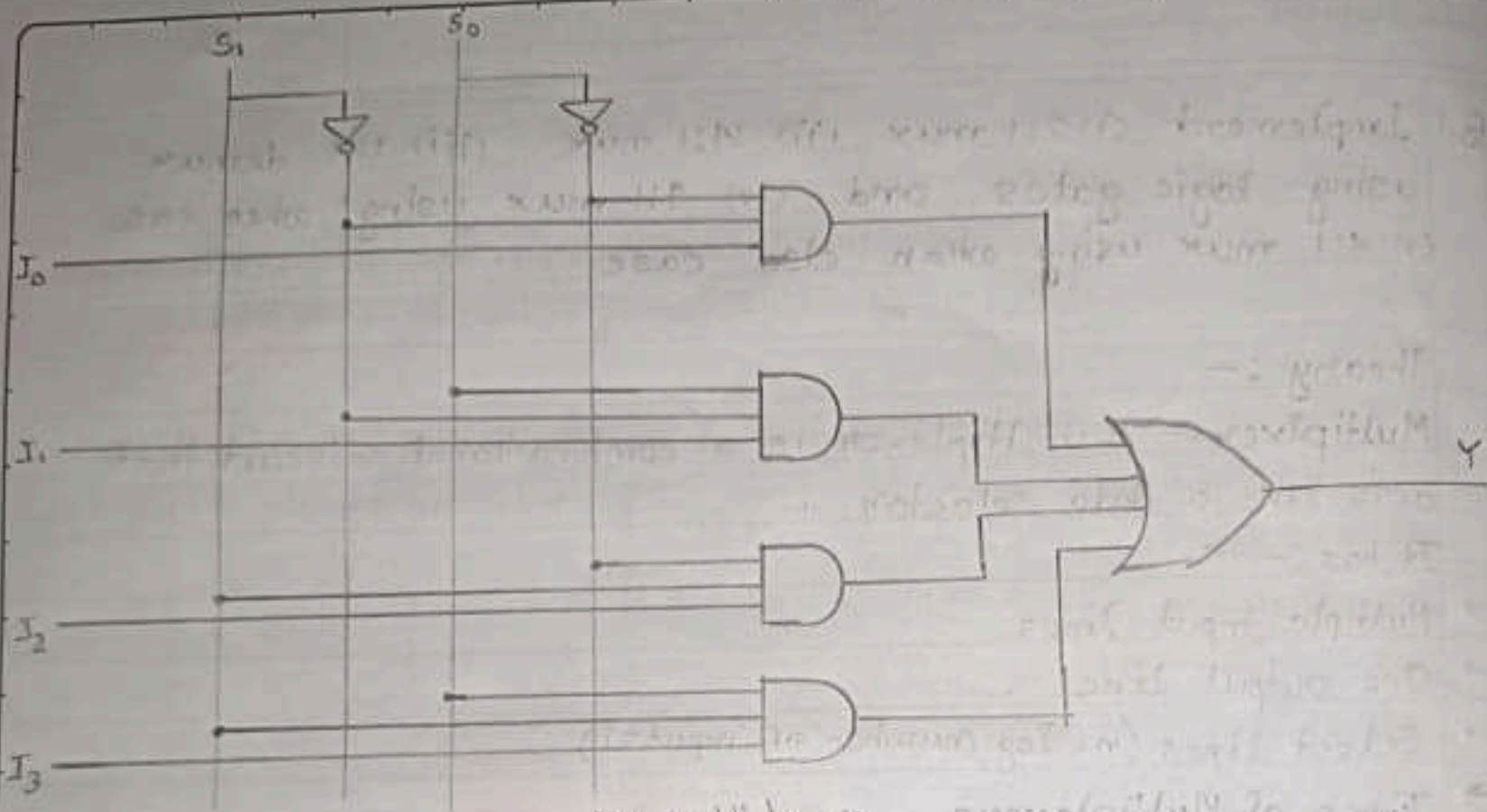
Y0 <= I AND nS1 AND nS0;

Y1 <= I AND nS1 AND S0;

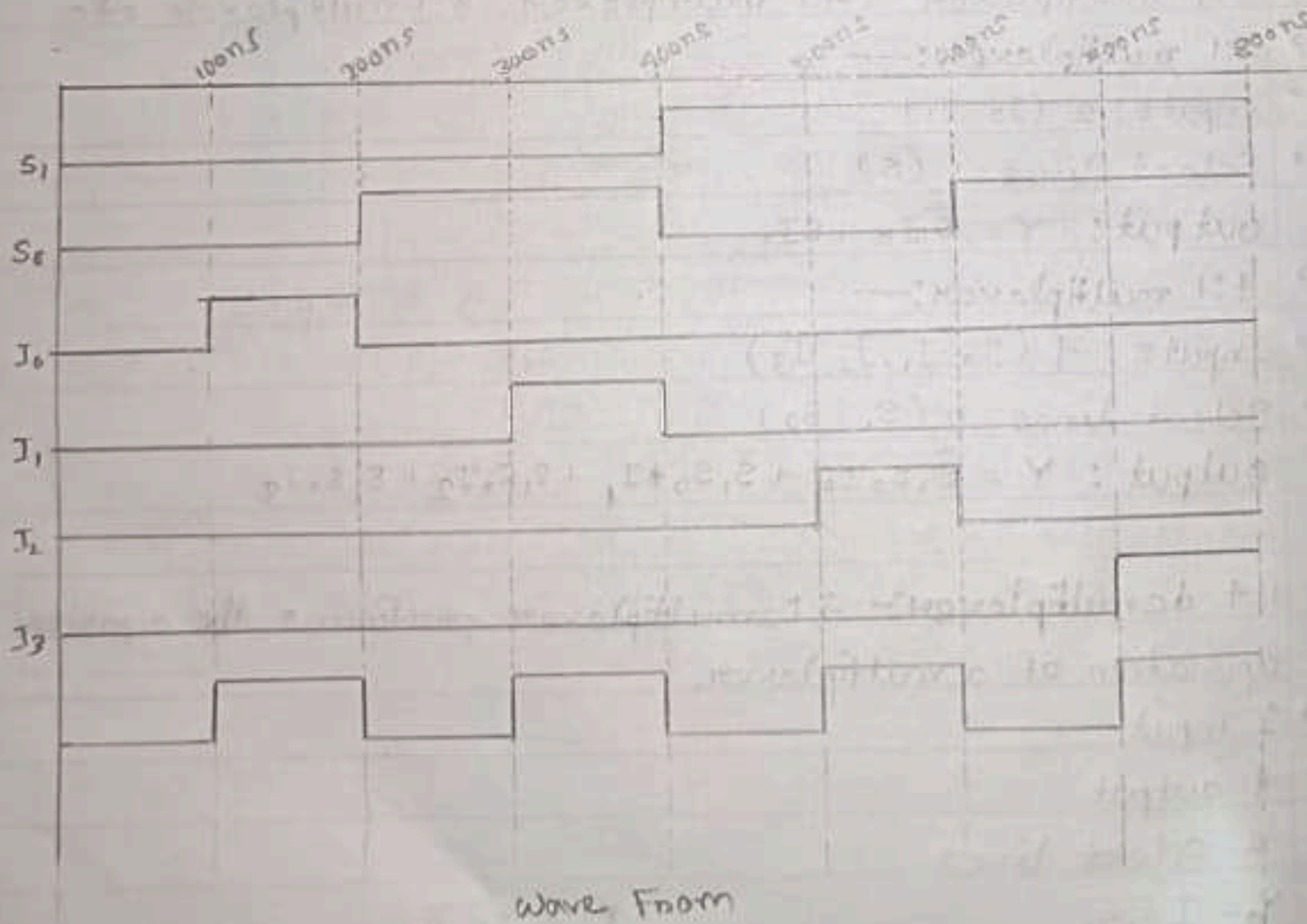
Y2 <= I AND S1 AND nS0;

Y3 <= I AND S1 AND S0;

end dataflow;



4:1 Multiplexer



VHDL Code:-

4:1 multiplexer:-

library IEEE;

use IEEE.STD_Logic_1164.ALL;

entity mux_4to1 is

port (I0: in STD_Logic;

I1: in STD_Logic;

I2: in STD_Logic;

I3: in STD_Logic;

S0: in STD_Logic;

S1: in STD_Logic);

Y: out STD_Logic);

end mux_4to1;

architecture dataflow of mux_4to1 is

signal n50, n51: STD_Logic;

signal A, B, C, D: STD_Logic;

begin

n50 <= NOT S0;

n51 <= NOT S1;

A <= I0 AND n51 AND n50;

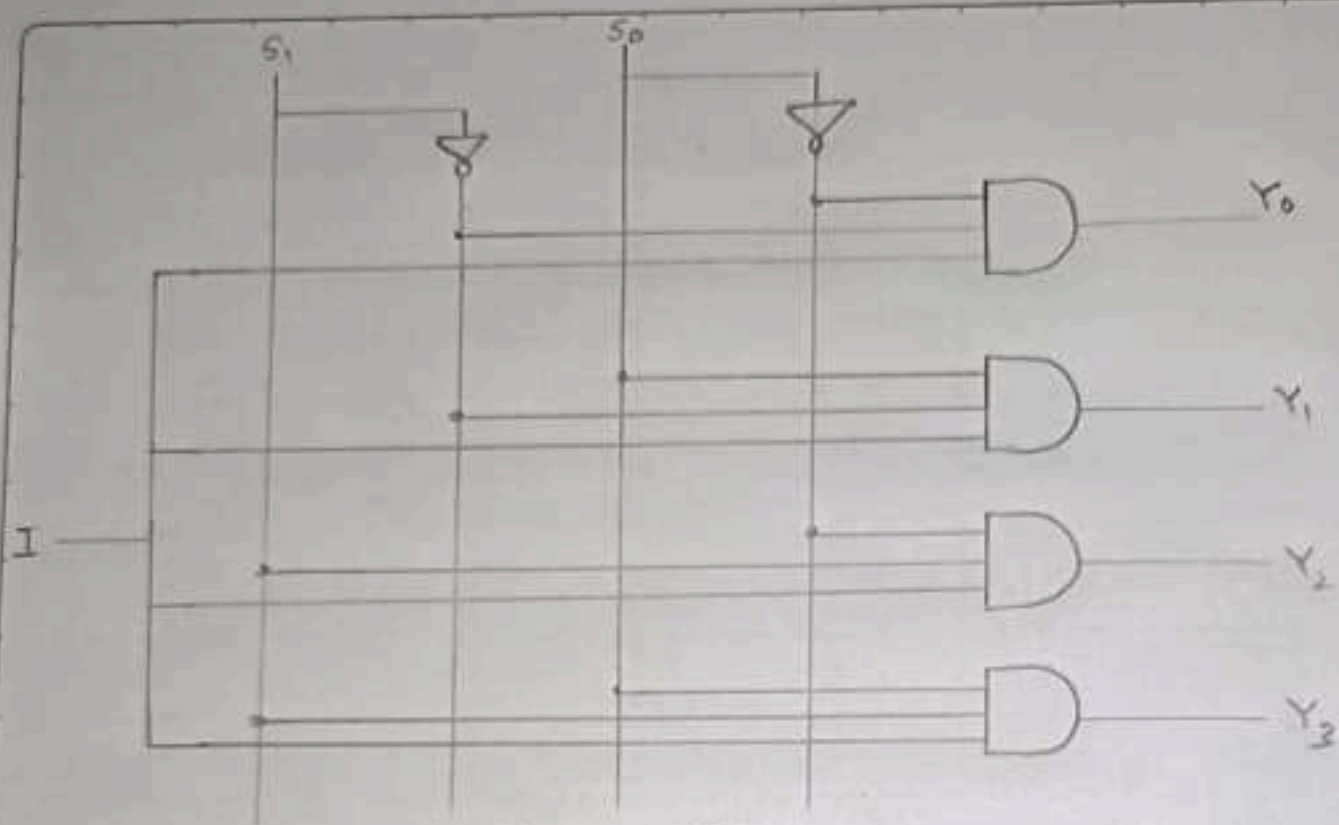
B <= I1 AND n51 AND S0;

C <= I2 AND S1 AND n50;

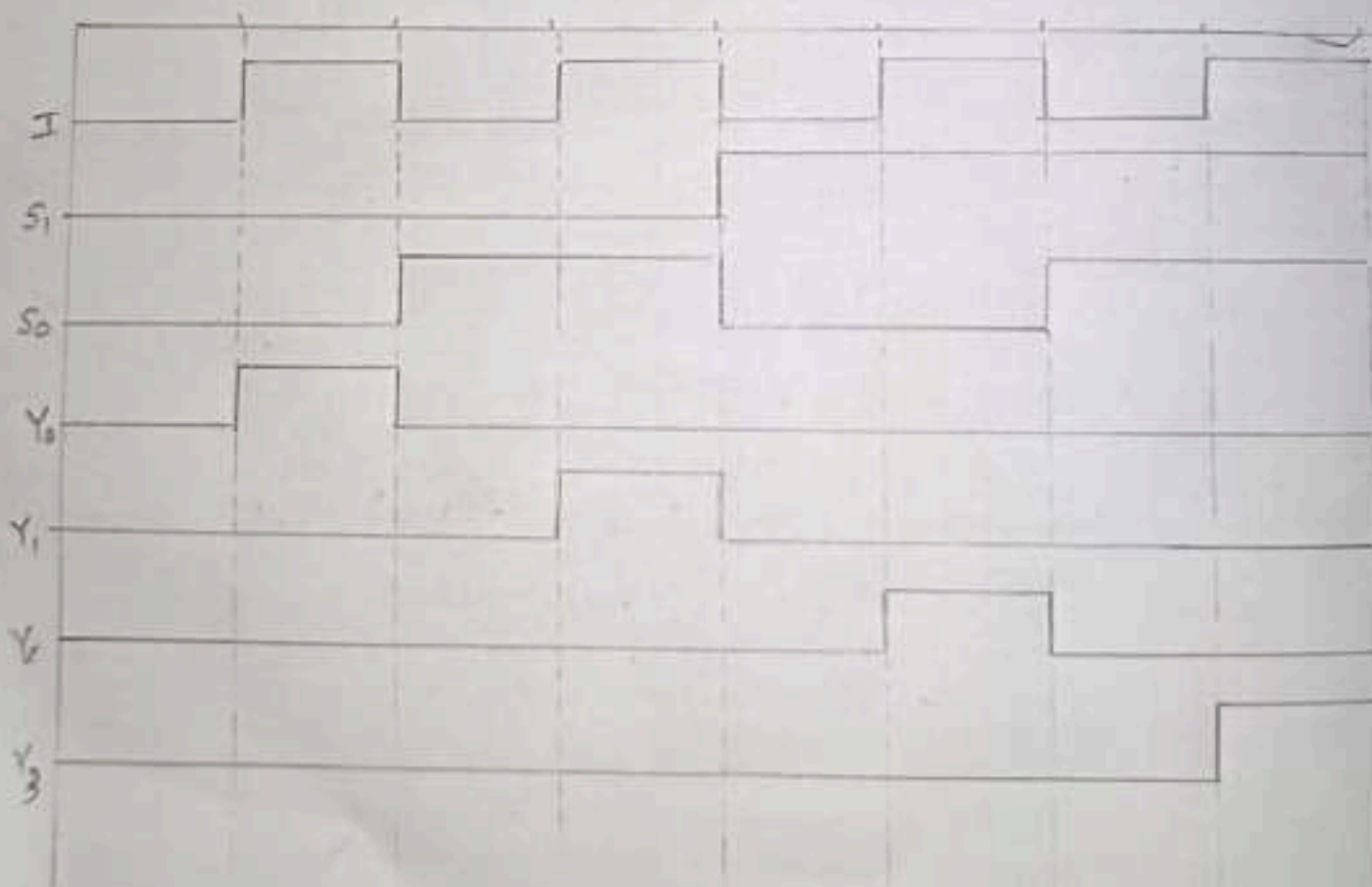
D <= I3 AND S1 AND S0;

Y <= A OR B OR C OR D;

end dataflow;



1:4 Demultiplexer



Wave Form

Q.1 Implement (i) 4:1 mux (ii) 2:1 mux (iii) 1:4 demux using logic gates and (iv) 4:1 mux using when case (v) 4:1 mux using when else case

Theory:-

Multiplexer:- A multiplexer is a combinational circuit that acts like a data selector.

It has:-

- Multiple input lines
- One output line
- Select lines ($n = \log_2(\text{number of inputs})$)

→ Types of Multiplexers

→ 2:1 multiplexer, 4:1 multiplexer, 8:1 multiplexer etc

→ 2:1 multiplexer:-

- Inputs: 2 (I_0, I_1)
- Select lines: 1 (S)

$$\text{output: } Y = \bar{S}I_0 + SI_1$$

→ 4:1 multiplexer:-

- Inputs: 4 (I_0, I_1, I_2, I_3)
- Select lines: 2 (S_1, S_0)

$$\text{output: } Y = \bar{S}_1\bar{S}_0I_0 + \bar{S}_1S_0I_1 + S_1\bar{S}_0I_2 + S_1S_0I_3$$

1:4 demultiplexer:- A Demultiplexer performs the opposite operation of a multiplexer.

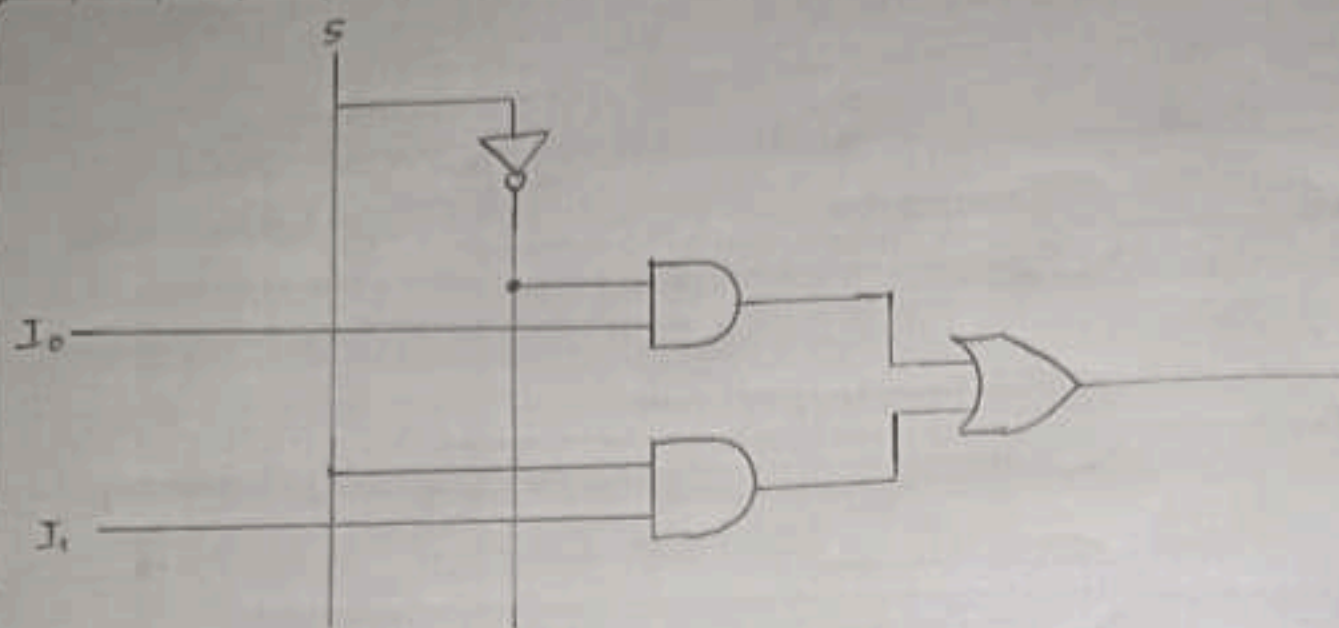
- 1 input
- 4 output
- 2 select lines

$$Y_0 = I\bar{S}_1\bar{S}_0$$

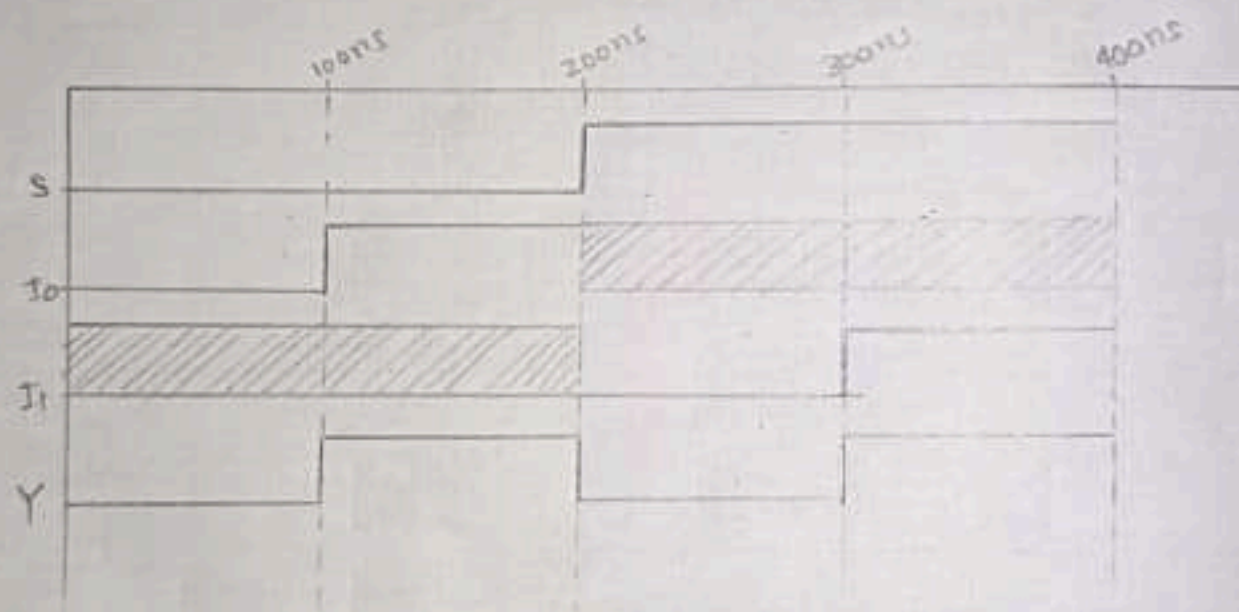
$$Y_1 = I\bar{S}_1S_0$$

$$Y_2 = IS_1\bar{S}_0$$

$$Y_3 = IS_1S_0$$



2:1 Multiplexer



Wave Form

Q.1 Implement (i) 4:1 mux (ii) 2:1 mux (iii) 1:4 demux using logic gates and (iv) 4:1 mux using when case (v) 4:1 mux using when else case

Theory:-

Multiplexer:- A multiplexer is a combinational circuit that acts like a data selector.

It has:-

- Multiple input lines
- One output line
- Select lines ($n = \log_2(\text{number of inputs})$)

→ **Types of Multiplexers**

→ 2:1 multiplexer, 4:1 multiplexer, 8:1 multiplexer etc

→ **2:1 multiplexer:-**

• Inputs: 2 (I_0, I_1)

• Select lines: 1 (S)

output: $Y = \bar{S}I_0 + SI_1$

→ **4:1 multiplexer:-**

• Inputs: 4 (I_0, I_1, I_2, I_3)

• Select lines: 2 (S_1, S_0)

output: $Y = \bar{S}_1\bar{S}_0I_0 + \bar{S}_1S_0I_1 + S_1\bar{S}_0I_2 + S_1S_0I_3$

1:4 demultiplexer:- A Demultiplexer performs the opposite operation of a multiplexer.

• 1 input

• 4 output

• 2 select lines

$$Y_0 = I\bar{S}_1\bar{S}_0$$

$$Y_1 = I\bar{S}_1S_0$$

$$Y_2 = IS_1\bar{S}_0$$

$$Y_3 = IS_1S_0$$

Subject / Important Note :

Page No. :

Date :

Expt. No. :

4:1 multiplexer using when else case:-

library IEEE;

use IEEE.STD_Logic_1164.ALL;

entity mux4to1 is

port (I0, I1, I2, I3 : in STD_Logic;

S1, S0 : in STD_Logic;

Y : out STD_Logic);

end mux4to1;

architecture dataflow of mux4to1 is begin

Y <= I0 when (S1='0' and S0='0') else

I1 when (S1='0' and S0='1') else

I2 when (S1='1' and S0='0') else

I3;

end dataflow;

Subject / Important Note :

Page No. :

Date :

Expt. No. :

4:1 multiplexer using when case:-

library IEEE;

use IEEE.STD_Logic_1164.ALL;

entity mux4to1 is

port (I0, I1, I2, I3 : in STD_Logic;

S1, S0 : in STD_Logic;

Y : out STD_Logic);

end mux4to1;

architecture dataflow of mux4to1 is begin

process (I0, I1, I2, I3, S1, S0) begin

case (S1 & S0) is

when "00" =>

Y <= I0;

when "01" =>

Y <= I1;

when "10" =>

Y <= I2;

when "11" =>

Y <= I3;

when others =>

Y <= '0';

end case;

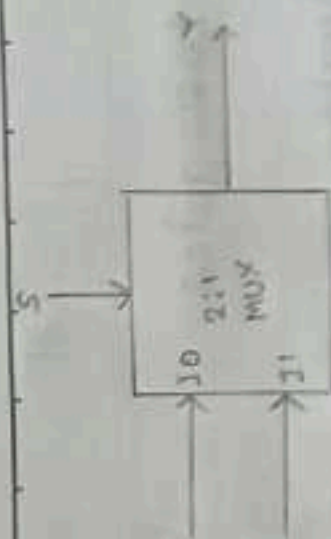
end process;

end dataflow;

Remarks :

Date :

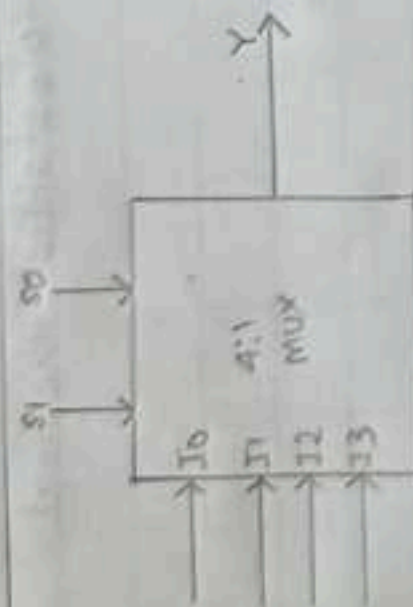
Teacher's Signature :



Block Diagram

S	I0	I1	Y
0	0	x	0
0	1	x	1
1	x	0	0
1	x	1	1

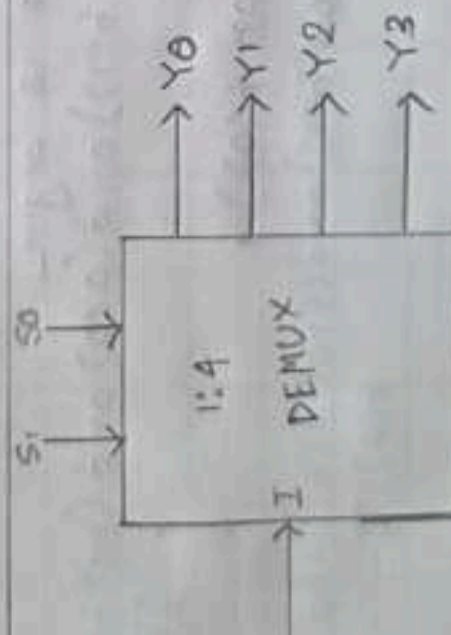
Truth Table



Block Diagram

S1	S0	I0	I1	I2	I3	Y
0	0	0	x	x	x	0
0	0	1	x	x	x	1
0	1	0	0	x	x	0
0	1	1	0	x	x	1
1	0	0	x	0	x	0
1	0	1	x	0	x	1
1	1	0	x	1	x	0
1	1	1	x	1	x	1

Truth Table



Block Diagram

I	S1	S0	Y0	Y1	Y2	Y3
0	0	0	0	0	0	0
1	0	0	1	0	0	0
0	0	1	0	1	0	0
1	0	1	0	0	1	0
0	1	0	0	0	0	1
1	1	0	0	0	0	0
0	1	1	0	0	0	0
1	1	1	0	0	0	0

Truth Table

2:1 multiplexer:-

library IEEE;

use IEEE, STD_Logic_1164.ALL;

entity mux2to1 is

port (I0: in STD_Logic;

I1: in STD_Logic;

S: in STD_Logic;

Y: out STD_Logic);

end mux2to1;

architecture dataflow of mux2to1 is

signal ns: STD_Logic;

signal A, B: STD_Logic;

ns <= 1

begin

ns <= NOT S;

A <= I0 AND ns;

B <= I1 AND S;

Y <= A OR B;

end dataflow;

AIM: Implement AND, OR and NOT gates using VHDL.

AND Gate;

Theory: AND gate is a basic digital logic gate that performs logical multiplication. It produces a HIGH (1) output when all inputs are HIGH (1). If any one of the inputs is Low (0), the output becomes Low (0).

It is commonly used in control and decision-making circuits.

Expression: $Y = A.B$

VHDL Code:

Library IEEE;

Use IEEE.STD_LOGIC_1164.ALL;

Entity and1 is

Port (inA1: in STD_LOGIC; -- AND GATE INPUT

inA2: in STD_LOGIC; -- AND GATE INPUT

OA: out STD_LOGIC); -- AND GATE OUTPUT

end and1;

architecture dataflow of and1 is

begin

OA <= inA1 and inA2; -- LOGIC OF 2 INPUT AND GATE

end dataflow;

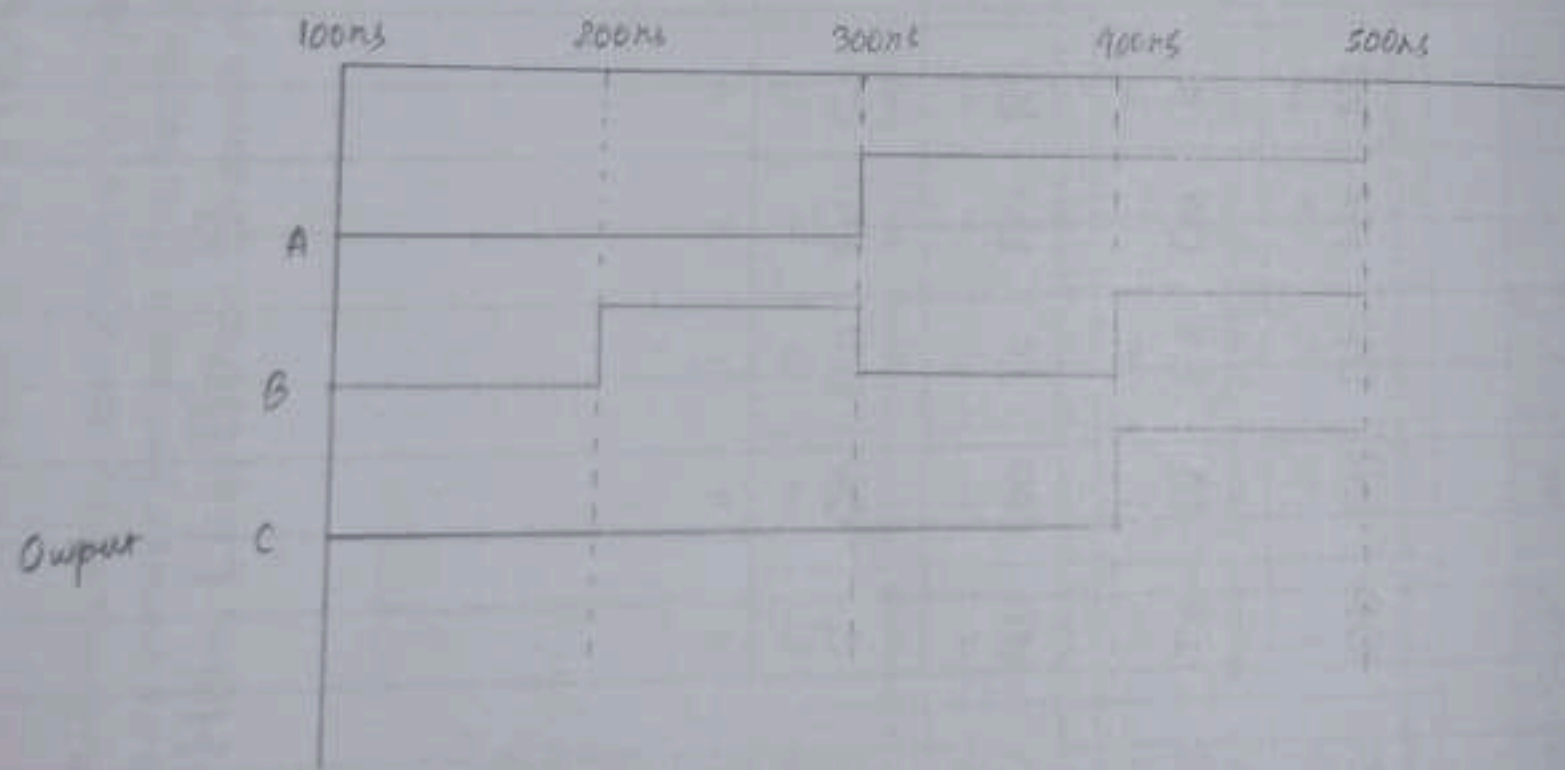
Teacher's Signature _____



Truth Table:

A	B	$Y = A \cdot B$
0	0	0
0	1	0
1	0	0
1	1	1

Wave Form:



Expt. No. _____

AIM: Implement AND, OR and NOT gates using V

AND Gate:

Theory: AND gate is a basic digital logic gate multiplication. It produces a HIGH (1) HIGH (1). If any one of the inputs is low (0).

It is commonly used in control a
Expression: $Y = A \cdot B$

VHDL Code:

Library IEEE;

Use IEEE.STD_LOGIC_1164.ALL;

Entity and1 is

Port (in A1: in STD_LOGIC; -- AND

in A2: in STD_LOGIC; -- AND

OA: out STD_LOGIC); -- AND

end and1;

architecture dataflow of and1 is

begin

OA <= in A1 and in A2; -- LOG

end dataflow;

OR Gate:

Theory: OR gate is a fundamental digital logic gate that performs logical addition. It produces a HIGH (1) output if atleast one of the inputs is HIGH (1). The output becomes Low (0) only when all inputs are Low (0). It is widely used in signal combining and decision circuits.

Expression: $Y = A + B$

VHDL Code:

Library IEEE;

Use IEEE.STD_LOGIC_1164.ALL;

Entity OR1 is

Port (inA1: in STD_LOGIC;

-- OR GATE INPUT

inA2: in STD_LOGIC;

-- OR GATE INPUT

OA: out STD_LOGIC);

-- OR GATE OUTPUT

end OR1;

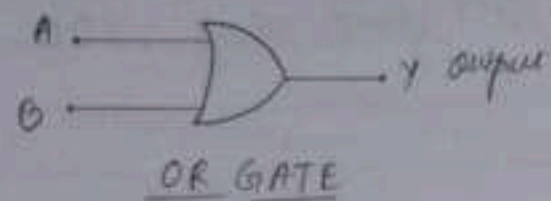
architecture dataflow of OR1 is

begin

OA <= inA1 or inA2;

-- LOGIC OF 2 INPUT OR GATE

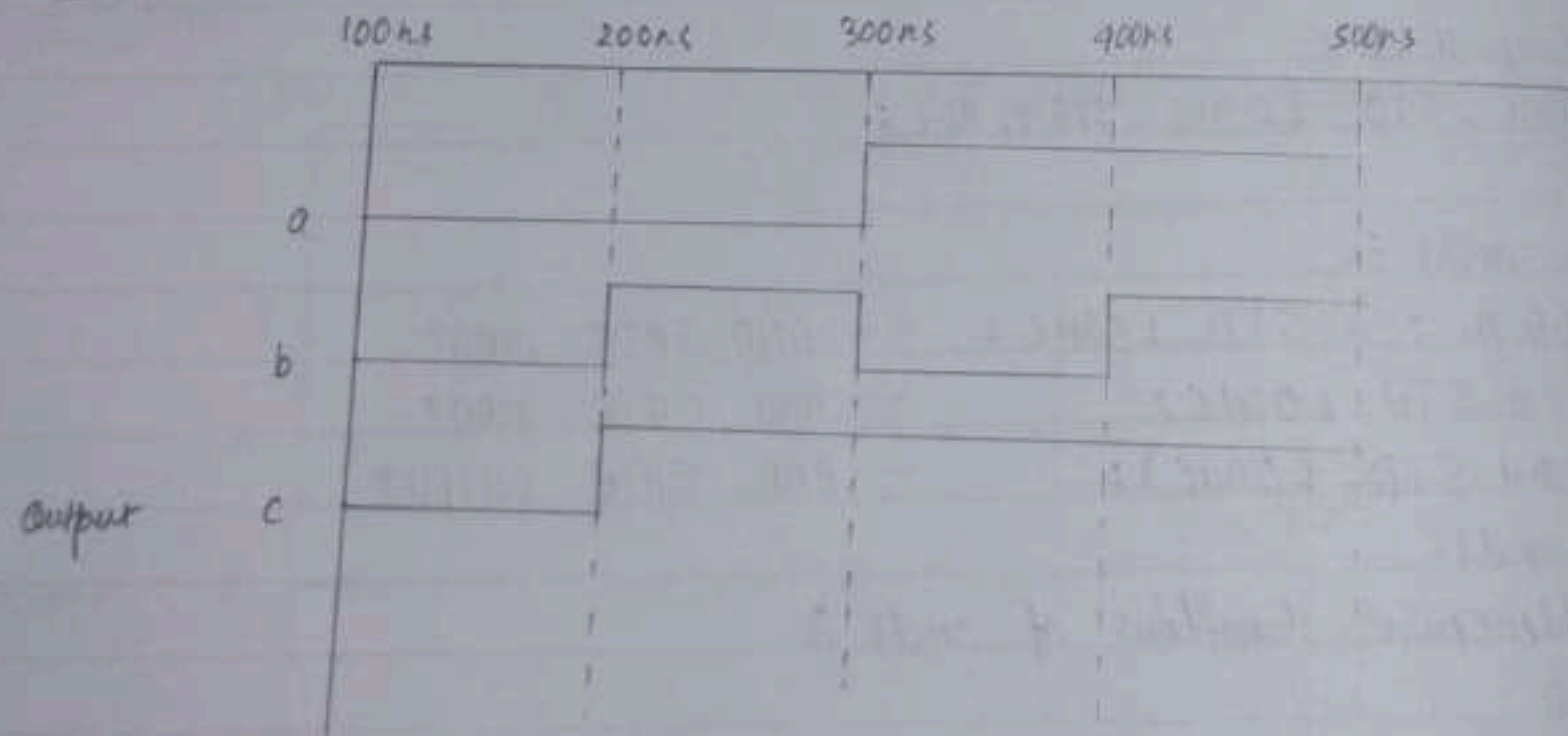
end dataflow;



Truth Table:

A	B	$Y = A + B$
0	0	0
0	1	1
1	0	1
1	1	1

Wave Form:



Expt. No. _____

OR Gate:

Theory: OR gate is a fundamental digital logic gate in addition. It produces a HIGH (1) output if at least one input is HIGH (1). The output becomes LOW (0) if all inputs are LOW (0). It is widely used in signal processing and decision circuits.

Expression: $Y = A + B$

VHDL Code:

Library IEEE;

Use IEEE.STD_LOGIC_1164.ALL;

Entity or1 is

Port (inA1: in STD_LOGIC; -- OR GATE

inA2: in STD_LOGIC; -- OR GATE

OA: out STD_LOGIC); -- OR GATE

end or1;

architecture dataflow of or1 is

begin

OA <= inA1 or inA2; -- LOGIC

end dataflow;

NOT Gate:

Theory: NOT gate, also an inverter is a digital logic gate that reverses the input signal. It produces the opposite of the given input. If the input is HIGH (1), the output LOW (0) and vice versa. It is mainly used for signal inversion operation.

VHDL Code:

Library IEEE;

Use IEEE.STD_LOGIC_1164.ALL;

Entity not1 is

Port (inA1 : in STD_LOGIC;

-- NOT GATE INPUT

OA : out STD_LOGIC);

-- NOT GATE OUTPUT

end not1;

architecture dataflow of not1 is

begin

OA <= not inA1;

-- LOGIC OF 1 INPUT NOT GATE

end dataflow;

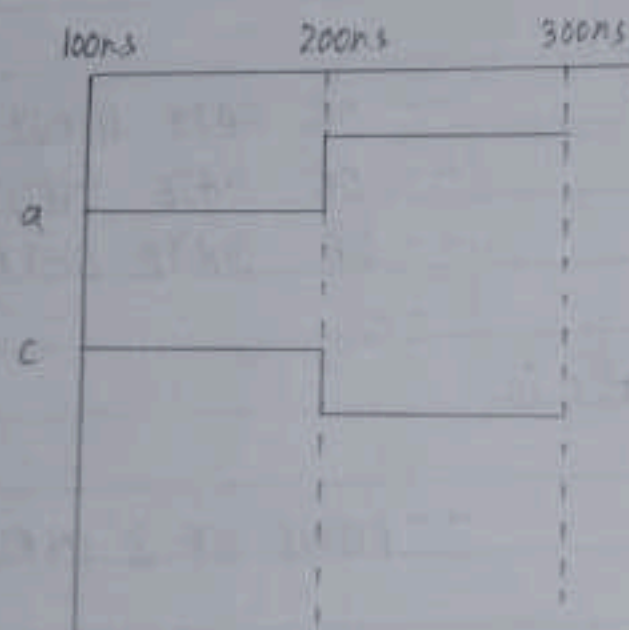


NOT GATE

Truth Table:

A	$Y = \bar{A}$
0	1
1	0

Wave Form:



NOT Gate:

Theory: NOT gate, also an inverter, reverses the input signal. If the input is given input. If the input is vice versa. It is mainly

VHDL Code:

Library IEEE;

Use IEEE.STD_LOGIC_1164.ALL;

Entity not1 is

Port (in A1 : in STD_LOGIC;

OA : out STD_LOGIC);

end not1;

architecture dataflow of not1

begin

OA <= not in A1;

end dataflow;